Art Gallery

Phase II: Threat Modeling

BIBIFI Group 11 | Huseyin Simsek and Gustavo Pepe

Secure Programming | Fall 2026

1. Project description

Our application keeps a record of who enters and leaves an art gallery and each room. Employees record movements at a check-in desk. Guests can see general room occupancy and only their own visit history. Administrators can manage people, login accounts, roles and audit records. Room occupancy is calculated from each person's latest accepted movement, rather than stored as a separate count.

We use Python 3.12, Flask, Flask-Login, Flask-WTF, Jinja templates, basic HTML/CSS and SQLite. The browser sends forms and page requests to Flask. The Flask routes check sessions and user roles, validate input and then read or update the SQLite file. There is no separate REST service or external database. Local setup creates the database and seeds sample records.

2. Important assets

Asset	Reason it needs protection
Accounts and roles	Usernames, password hashes and access levels determine who can use protected features.
Session and local secrets	The signed login cookie and SECRET_KEY protect sessions; CSRF tokens protect write requests.
Persons and visit history	Visitor names and movement histories can disclose where individuals were and when.
Gallery event records	Event order, person IDs, room IDs and recorder IDs determine the current state of the gallery.
Room configuration	Room names and capacities are needed to reject unsafe or impossible movements.
Audit records	Login, access-denial and movement logs help attribute actions and investigate problems.
SQLite file and application	The file stores all application records. Loss, tampering or downtime affects the entire system.

The highest-value data are staff and administrator accounts, signed sessions, the event history, and the SQLite database itself. A mistake affecting one of these can change who has access or whether visit records can be trusted.

3. Threat actors

Actor	What they may try
Outside attacker	Guess passwords, steal a session cookie, send malicious inputs or overload public endpoints.
Guest user	Change a URL parameter to see someone else's history or call a staff route directly.
Employee account	Record false movements, submit repeated entries or try administrator operations.
Administrator	Create inappropriate privileges, associate an account with the wrong person or misuse full access.
Someone with file access	Read, change or delete local configuration, the SQLite database or audit information.

4. System components

Component	Function and security concern
Browser and Jinja pages	Collect form fields, display results and pass a signed session cookie. The browser cannot be trusted to enforce roles.
Login and sessions	Check password hashes, load accounts from SQLite, maintain Flask-Login sessions and enforce login limits.
Gallery event routes	Validate person, room, event type and previous location; reject invalid changes and full rooms.
History and occupancy routes	Reconstruct state from events, filter history and restrict a guest to their linked person.
Administrator routes	Create people and accounts, change roles and display the audit log.
SQLite and setup script	Store six tables and initialize local demonstration records. SQLite has no separate app/database users.

Trust boundaries

TB1: Browser to Flask. Cookies, form fields, hidden values, record IDs and query strings all arrive from the browser. Flask must validate them and check permissions before acting. A hidden navigation link is not an authorization rule.

TB2: Flask process to SQLite file. The application executes SQL through Python's sqlite3 library. Query parameters reduce SQL injection risk, but someone with local access to the database file could bypass all application checks.

Local setup. The setup script and CLI commands are operated by a trusted developer. They generate private configuration and create/seed database records. They are not browser endpoints.

5. How the threats were identified

We started with the implemented routes and the data they accept. The login form accepts credentials. The event form accepts a person ID, event type and optional room ID. History accepts filters and page numbers. Administrator forms accept account and role information. We considered what would happen if an outside attacker, guest, employee or administrator supplied unexpected values or attempted an action outside their permissions.

For each entry point, we checked the path from the browser through Flask to the SQLite tables. We considered the six STRIDE categories and recorded the part of the application affected, a concrete attack scenario, what data or functionality would be harmed, and whether the current code contains a relevant safeguard. We then assigned impact and likelihood scores using the method on page 5. The threat register at the end contains the complete set of twenty cases.

6. STRIDE analysis

Spoofing: login and session handling

A staff account is a more valuable target than a guest account because it can record gallery movements. An administrator can also create and modify users. A password-guessing attack therefore has a potentially serious effect. The login route keeps recent failures by username and by IP address and returns an error after the defined thresholds. Passwords are checked against scrypt hashes rather than stored in plaintext. This reduces repeated guessing but does not remove risks from reused or stolen passwords (T01).

Flask-Login maintains a signed session cookie. The cookie is HttpOnly and SameSite=Lax, and the configured session lifetime is 30 minutes. Those controls do not encrypt traffic. The current application sets SESSION_COOKIE_SECURE=False because it is a local project. If deployed without HTTPS, an intercepted cookie could be reused. TLS and Secure cookies are necessary before public hosting (T02).

Tampering: movements, forms and SQL

A visitor should move outside, then onto the gallery floor, then into a room, and back out in the reverse order. The event route checks the person's last recorded state, room existence and capacity. It rejects duplicates, attempts to leave a different room and attempts to leave the gallery while inside a room. Invalid events return HTTP 409 and do not create a movement row (T03).

Two employees may submit events for the same room or person at nearly the same time. The code uses BEGIN IMMEDIATE before calculating the latest state and writing the event. This serializes SQLite writers, preventing two simultaneous requests from both accepting an outdated capacity count (T04). Flask-WTF protects form submissions with CSRF tokens (T05), and SQLite values are bound with ? parameters after basic input validation (T06).

Repudiation: identifying who performed an action

Each gallery event stores recorded_by from the signed-in user rather than trusting an account ID submitted by the browser. The audit table records the actor, action, time, target, outcome and IP for several security-relevant routes (T07). This helps during a review, but the logs are stored in the same SQLite file as the main data. Anyone with direct file modification access could alter both movement records and audit evidence (T08). Some failures may also have incomplete audit coverage, so server error handling and log retention need further review (T09).

6. STRIDE analysis (continued)

Information disclosure: visit history and application data

A guest can include a different person_id in a history URL. The history route checks the ID against the account's linked person record and returns HTTP 403 when the request is unauthorized. A guest with no linked person receives an empty history result. This is a direct control against viewing another visitor's movements (T10).

The application can display names that were entered by an administrator. Jinja escapes ordinary template values, and the Content Security Policy disables script execution. These controls reduce the risk of stored script injection through a person name (T11). Private settings and the database are kept outside the static directory and excluded from Git. Those controls do not protect copies or backups saved elsewhere (T12). The database enforces a unique users.person_id link, but an administrator could still choose the wrong valid person (T13).

Denial of service: login, queries and local storage

The login route checks recent failed attempts, but its failure table can continue to grow without an expiration cleanup. Automated requests could increase storage use or slow the database (T14). The history route limits requested page sizes to 25, and indexes support event lookups; occupancy still reconstructs the current state from the event table. Performance testing would be needed with much larger histories (T15). Because the application uses one local SQLite file, deleting or corrupting that file could stop normal operations. A tested backup and recovery procedure is needed (T16).

Elevation of privilege: route and account permissions

A guest or employee may try to reach /admin or issue a direct POST to an administrator endpoint. The role_required decorator checks the role loaded from SQLite, not a value supplied in the request. Administrator actions accept only a limited set of form fields; role changes reject self-demotion and attempts to remove the last administrator (T17 and T18).

A legitimate administrator still has the ability to create additional privileged accounts or change someone's role. The application records those actions, but it does not require a second administrator to approve them. This is an insider risk rather than a bypass of the role checks (T19).

Code locations reviewed

The main controls described above are implemented in **src/backend/routes.py**, **src/backend/__init__.py**, **src/backend/db.py** and **database/schema.sql**. The security tests are in **tests/security/test_security.py**. The application was reviewed at GitHub commit **d4444bd** on the main branch.

7. Risk scoring

We used a simple three-point scale for impact and likelihood. Impact is the harm if the attack succeeds; likelihood is our qualitative estimate given the current implementation and expected local use. These are judgments from the code review, not measured incident rates.

Score	Impact	Likelihood
1	Low: a limited or recoverable effect	Unlikely: existing checks make the attack difficult in the stated setting
2	Moderate: loss of a feature, partial incorrect data or limited disclosure	Possible: an error, misuse or repeated attempt could reach the condition
3	High: administrator compromise, significant disclosure or loss of main records	Likely: the exposure is broad and current controls offer little resistance

For every threat, **risk score = likelihood x impact**. A score of 1 or 2 is Low, 3 or 4 is Medium, and 6 or 9 is High. This gives a consistent reason for the High, Medium and Low labels in the register.

Risk matrix

Likelihood / Impact	Low (1)	Moderate (2)	High (3)
Likely (3)	3: Medium	6: High	9: High
Possible (2)	2: Low	4: Medium	6: High
Unlikely (1)	1: Low	2: Low	3: Medium

Highest-priority risks

Threat	L x I	Why we rated it High
T01: Staff password guessing	2 x 3 = 6	Compromise of a staff or administrator account affects movement integrity or account management.
T02: Stolen session	2 x 3 = 6	A valid cookie can provide the role and access of the user whose session is stolen.
T08: Audit record changes	2 x 3 = 6	An attacker with local file access can erase evidence along with the main records.
T12: Private file exposure	2 x 3 = 6	A disclosed database or secret key can expose records or compromise sessions.
T16: Database loss	2 x 3 = 6	A damaged SQLite file makes central features unavailable.
T19: Administrator misuse	2 x 3 = 6	A legitimate privileged user can create accounts or change access.
T20: Unsafe public deployment	2 x 3 = 6	Internet-facing HTTP and non-Secure cookies introduce session exposure.

The remaining risks are mainly Medium. SQL parameter binding and strict ID parsing make T06 the only Low-rated scenario in this assessment. Ratings would change if the application were publicly deployed or had real visitor data.

8. Mitigation plan

Some controls are already implemented. Others are next steps, particularly if the application is deployed beyond the local course environment. The table identifies the change and where it belongs.

Priority	Action and implementation	Risks addressed
1	**Protect any network deployment.** Put Flask behind HTTPS, set SESSION_COOKIE_SECURE=True, and verify configuration is loaded only from private files. Keep public debug output disabled.	T02, T12, T20
2	**Protect and back up data.** Restrict access to instance/week6.db. Back up the file and test a clean restoration. Keep an audit copy outside the editable application database.	T08, T12, T16
3	**Review privileged accounts.** Review new admin users and role changes. Consider MFA for administrator login and require team approval for sensitive account changes.	T01, T17, T18, T19
4	**Limit repeated requests.** Delete old login_failures records, measure history and occupancy performance as the event table grows, and add broader HTTP rate limits if needed.	T14, T15
5	**Preserve validation checks.** Keep tests for invalid movements, full rooms, concurrent entries, missing CSRF tokens, authorization by role and guest history privacy.	T03, T04, T05, T06, T10, T11
6	**Review audit coverage.** Document which requests append audit rows and confirm server errors do not silently lose necessary evidence.	T07, T08, T09

9. Evidence and limitations

The repository contains functional and security tests for login, roles, invalid movement order, room capacity, CSRF, malformed inputs, HTML escaping and concurrent movements. These tests are evidence of the intended controls, not evidence of a penetration test. The instructor's correctness oracle was not provided to us, and this review did not rerun the latest repository test suite or deploy the site on a public server.

The application is a local Flask and SQLite project. Threat ratings assume normal course use on a trusted machine. They should be reconsidered if it stores personal data from real visitors, accepts traffic from the Internet or runs with different file permissions.

Conclusion

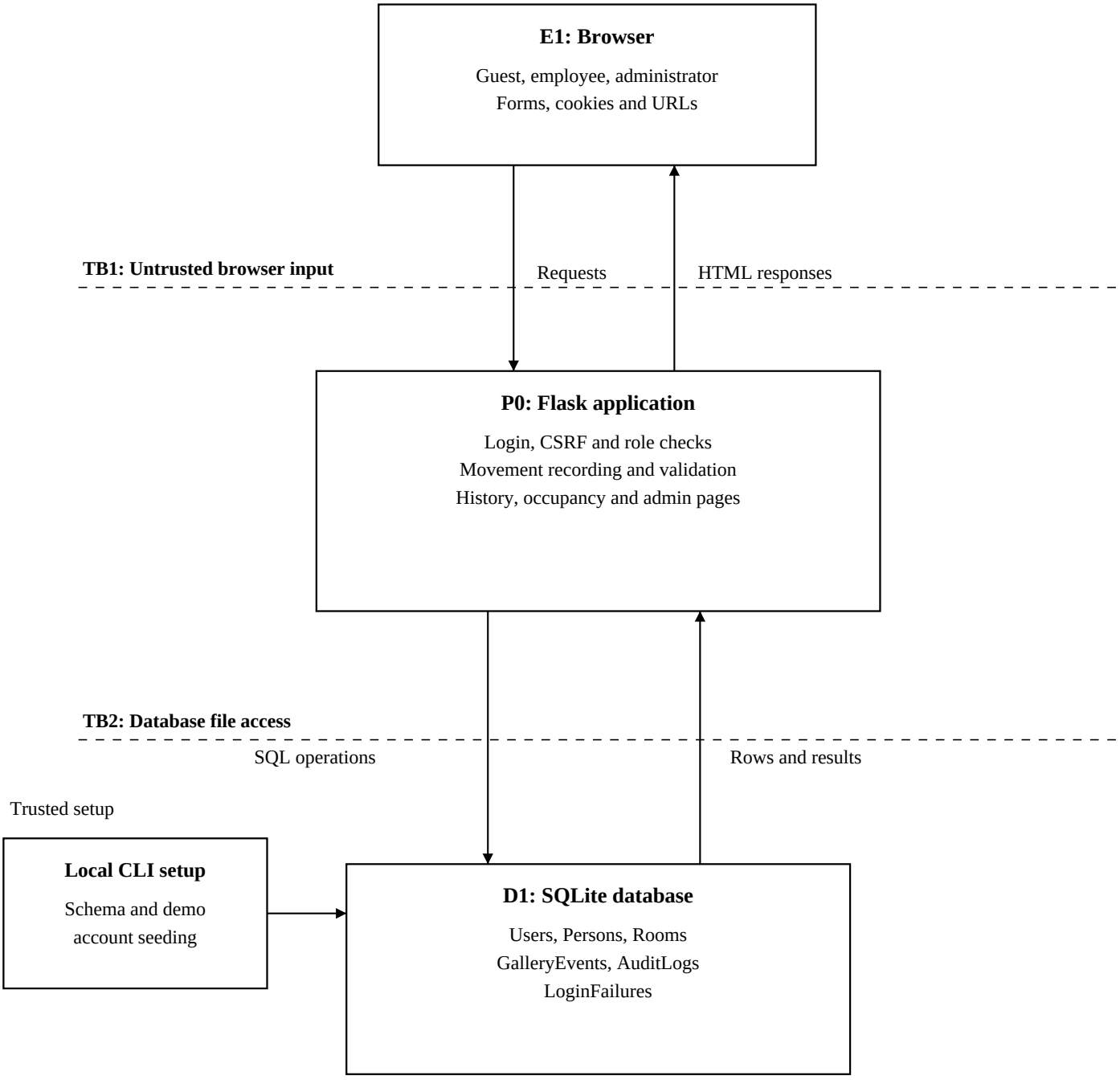
The main security boundaries are the browser-to-Flask request and the Flask-to-SQLite file access. The code already performs role checks, validates movement order, prevents room overcapacity and restricts guest history. The most important work remaining for a real deployment is transport security, database backup and recovery, stronger audit-file protection and review of administrator access.

Source reviewed: github.com/hsimsek1/art-gallery-group11, main branch, commit d4444bd24bf7dc5a08c17665e0e5a78322246beb (October 30, 2026).

Art Gallery

Data flow diagram: Level 0

BIBIFI Group 11



TB1: Flask checks sessions, user roles, CSRF tokens and form values.

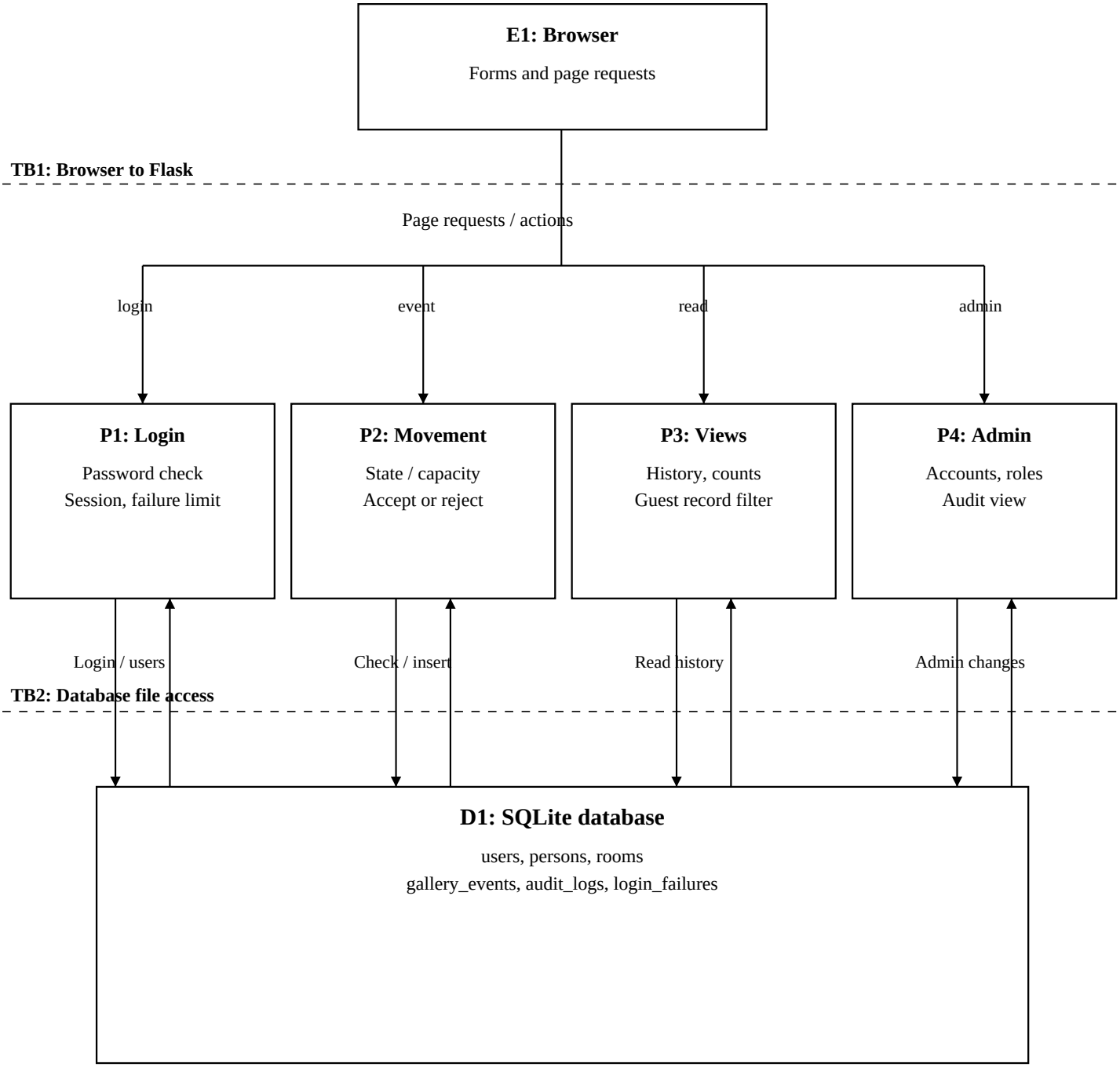
TB2: SQLite is not directly accessible to web users.

Someone with file-system access may bypass application checks.

The local setup command creates and seeds the database.

Art Gallery

Data flow diagram: Level 1



The server checks requested movements before inserting GalleryEvents.

A rejected movement adds an audit row, not an event record.

Successful writes include the authenticated recorder and server timestamp.

Queries return rows for HTML pages; failed requests return an error response.

The browser cannot access SQLite directly.

Art Gallery

STRIDE Threat Register

BIBIFI Group 11 | Phase II

L = likelihood (1 to 3). I = impact (1 to 3). Risk = L x I. Low: 1 to 2; Medium: 3 to 4; High: 6 to 9. The current control and recommended action are listed separately.

ID / STRIDE	Area and scenario	L / I / Risk	Control and action
T01 Spoofing	**Login** An attacker repeatedly guesses an employee or administrator password.	2 / 3 6 High	**Present:** Scrypt password hashes; generic login errors; five failures per username and twenty per IP in fifteen minutes. **Next:** Review administrator MFA; monitor repeated failures and unusual sources.
T02 Spoofing	**Session** A stolen session cookie lets someone act as a logged-in account.	2 / 3 6 High	**Present:** HttpOnly and SameSite=Lax cookies; session cleared at login/logout; 30-minute configured lifetime. **Next:** Use HTTPS and Secure cookies before allowing remote connections.
T03 Tampering	**Event order** An employee records the same entry twice or records a room exit from the wrong room.	2 / 2 4 Medium	**Present:** Server reconstructs a person's latest state and rejects impossible movements with HTTP 409. **Next:** Retain tests for all four movements and invalid sequences.
T04 Tampering	**Capacity** Two near-simultaneous requests attempt to enter a room with only one remaining place.	2 / 2 4 Medium	**Present:** State and capacity checks run inside a SQLite BEGIN IMMEDIATE write transaction. **Next:** Repeat concurrency tests on the deployment configuration.
T05 Tampering	**CSRF** A hostile site tries to make a signed-in employee submit an unwanted movement.	1 / 3 3 Medium	**Present:** Flask-WTF CSRF checks and POST-only state changes. **Next:** Check the token requirement on every new write route.
T06 Tampering	**SQL input** Crafted filter or identifier values attempt to change a query.	1 / 2 2 Low	**Present:** Integer checks, enumerated event types, and bound SQLite query parameters. **Next:** Review any newly added queries and form fields before release.
T07 Repudiation	**Recorded events** A staff member denies creating a specific movement record.	2 / 2 4 Medium	**Present:** Event records store recorded_by from the authenticated account; audit rows include actor and time. **Next:** Protect account access and document how staff activity is reviewed.
T08 Repudiation	**Audit records** Someone with file-system access edits or deletes audit rows.	2 / 3 6 High	**Present:** Audit data exists in SQLite with action, target, outcome, user, time and IP. **Next:** Restrict database-file access and export logs to a separately protected destination.
T09 Repudiation	**Error coverage** A failure happens without enough information being kept for later review.	2 / 2 4 Medium	**Present:** Login attempts, denied access and accepted/rejected gallery events are logged. **Next:** Review error cases, logging failures and server exceptions.
T10 Information disclosure	**History** A guest changes a person_id query parameter to read another visitor's history.	1 / 3 3 Medium	**Present:** Guests are restricted to the person linked to their account; unauthorized person filters return 403. **Next:** Preserve the restriction in each history endpoint and regression test.
T11 Information disclosure	**HTML output** A person name containing script markup is displayed in a page.	1 / 3 3 Medium	**Present:** Jinja escapes user text; the content security policy blocks scripts. **Next:** Avoid unescaped HTML and recheck output whenever templates change.

ID / STRIDE	Area and scenario	L / I / Risk	Control and action
T12 Information disclosure	**Local files** A secret key or database copy becomes publicly accessible.	2 / 3 6 High	**Present:** The SQLite file and local settings are outside static assets and excluded from Git. **Next:** Check repository history, apply OS permissions and encrypt off-machine backups.
T13 Information disclosure	**Guest linking** An administrator links a guest account to the wrong person record.	2 / 2 4 Medium	**Present:** The users.person_id relationship is unique and enforced by a foreign key. **Next:** Use a second check before confirming an account-person association.
T14 Denial of service	**Login failures** Automated login attempts create many failure records and consume server resources.	2 / 2 4 Medium	**Present:** Per-IP and per-username login failure checks limit repeated authentication attempts. **Next:** Prune old login_failures rows and add request throttling at the server boundary.
T15 Denial of service	**Read routes** Large event histories or repeated requests slow gallery and history views.	2 / 2 4 Medium	**Present:** History uses bounded pages of at most 25 entries and indexed event columns. **Next:** Measure read times at realistic data sizes and set overall request limits.
T16 Denial of service	**Database file** A damaged or deleted SQLite file makes visit history and login records unavailable.	2 / 3 6 High	**Present:** Database is a local file separate from public web assets. **Next:** Schedule backups, test restoration and restrict modification rights.
T17 Elevation of privilege	**Admin endpoints** A guest or employee sends a direct request to /admin or /admin/users.	1 / 3 3 Medium	**Present:** Route decorators check the role stored in SQLite and reject unauthorized access. **Next:** Test each protected route for every role after changes.
T18 Elevation of privilege	**Role changes** A user submits an extra role field or tries to demote their own admin account.	1 / 3 3 Medium	**Present:** Only approved form fields are accepted; role changes are admin-only; self-change is blocked. **Next:** Review all role-change actions and keep negative tests.
T19 Elevation of privilege	**Admin misuse** A legitimate administrator creates a new privileged user without team approval.	2 / 3 6 High	**Present:** The application records administrator account creation and role changes. **Next:** Review admin accounts regularly and require approval outside the app for sensitive changes.
T20 Information disclosure	**Deployment** Someone exposes the local HTTP server over a public connection.	2 / 3 6 High	**Present:** Docker Compose binds the published port to loopback, not all network interfaces. **Next:** Configure HTTPS, Secure cookies, secrets handling and a deployment review before public hosting.